Overall Intro to Say to External

My practicals are divided into two parts:

1. **HPC Practical using OpenMP and C++**
    - Parallel BFS and DFS
    - Parallel Bubble Sort and Merge Sort
    - Parallel Reduction for Min, Max, Sum, Average
2. **Deep Learning Practical using Python, TensorFlow/Keras**
    - Boston Housing Price Prediction using DNN
    - IMDB Movie Review Binary Classification using DNN
    - Google Stock Price Prediction using RNN

In HPC programs, I used **OpenMP** for parallel execution. OpenMP creates multiple threads and executes independent parts of the program simultaneously. In Deep Learning programs, I used **TensorFlow/Keras** to build neural networks and train them on datasets.

Practical 1: Parallel BFS and DFS using OpenMP

Aim

To implement **Breadth First Search** and **Depth First Search** on a graph using both sequential and parallel methods with OpenMP.

Concept

A graph contains **vertices/nodes** and **edges**. In the code, the graph is stored using an **adjacency list**.

Example:

vector<vector<int>> adj;

This means each vertex stores a list of its connected neighbours.

The graph used in the program has 7 vertices:

0-1, 0-2, 1-3, 1-4, 2-5, 2-6

So, it looks like a tree:

```
      0
    /   \
   1     2
  / \   / \
 3   4 5   6
```

Important Terms

V means **number of vertices**.

int V = 7;

visited array is used to check whether a node is already visited or not.

vector<bool> visited(V, false);

Initially all values are false. When a node is visited, it becomes true.

Sequential BFS Working

BFS means **Breadth First Search**.

It visits nodes level by level.

For example, starting from 0:

0 → 1 → 2 → 3 → 4 → 5 → 6

It uses a **queue** because BFS follows FIFO: First In First Out.

Steps:

1. Start from source node.
2. Mark it as visited.
3. Push it into queue.
4. Remove front node from queue.
5. Visit its unvisited neighbours.
6. Push neighbours into queue.
7. Repeat until queue becomes empty.

Code concept:

```
queue<int> q;
visited[start] = true;
q.push(start);
```

Parallel BFS Working

Parallel BFS is done **level-wise** using the concept of **frontier**.

frontier stores nodes of current level.

For example:

Level 0: 0

Level 1: 1, 2

Level 2: 3, 4, 5, 6

Nodes on the same level can be processed in parallel because they are independent.

Code uses:

```
#pragma omp parallel
#pragma omp for nowait
```

Each thread processes some nodes from the current frontier.

The next level nodes are stored in next_frontier.

A critical section is used while updating visited, because multiple threads may try to visit the same node at the same time.

```
#pragma omp critical
{
   if (!visited[nbr]) {
      visited[nbr] = true;
      added = true;
   }
}
```

This avoids a **race condition**.

Sequential DFS Working

DFS means **Depth First Search**.

It goes deep into one path first, then backtracks.

The program uses a **stack**.

DFS starting from 0 may visit:

0 → 1 → 3 → 4 → 2 → 5 → 6

It uses stack because DFS follows LIFO: Last In First Out.

Steps:

1. Push starting node into stack.
2. Pop top node.
3. If not visited, visit it.
4. Push its unvisited neighbours.
5. Continue until stack becomes empty.

Parallel DFS Working

Parallel DFS uses recursive function:

parallelDFSUtil()

In this function:

1. A node is checked inside critical section.
2. If not visited, it is marked visited.
3. Its neighbours are explored in parallel using:

```
#pragma omp parallel for
```

The main parallel DFS starts with:

```
#pragma omp parallel
{
   #pragma omp single
   {
      parallelDFSUtil(start, visited);
   }
}
```

single means only one thread starts DFS initially, then further recursive calls can explore neighbours in parallel.

Output Meaning

The program prints:

- Adjacency list
- Sequential BFS result
- Parallel BFS result
- Sequential DFS result
- Parallel DFS result
- Time taken by each method

Time is calculated using:

omp_get_wtime()

Speed may not always be better in parallel for small graphs because thread creation and synchronization also take time.

Short Exam Writeup

Parallel BFS and DFS using OpenMP:

This practical implements BFS and DFS traversal on an undirected graph using sequential and parallel approaches. The graph is represented using an adjacency list. Sequential BFS uses a queue and visits nodes level by level, while sequential DFS uses a stack and explores nodes deeply before backtracking. Parallel BFS uses a frontier-based approach where all nodes at the same level are processed simultaneously using OpenMP threads. Parallel DFS uses recursive exploration of neighbouring nodes with OpenMP parallel loops. The visited array prevents repeated traversal, and critical sections are used to avoid race conditions. Execution time is measured using omp_get_wtime() to compare sequential and parallel performance.

Viva Points

External may ask:

Q. What is BFS?

BFS is Breadth First Search. It visits graph nodes level by level using a queue.

Q. What is DFS?

DFS is Depth First Search. It visits one path deeply before backtracking using stack or recursion.

Q. Why is visited used?

To avoid visiting the same node again and to prevent infinite loops.

Q. Why is critical used?

Because multiple threads may try to update visited at the same time. Critical section prevents race condition.

Q. Why parallel BFS is easier than parallel DFS?

Because BFS works level by level, and nodes of the same level can be processed independently.

Q. Why parallel output order may differ?

Because multiple threads execute at the same time, so order is not always fixed.

Practical 2: Parallel Bubble Sort and Merge Sort using OpenMP

Aim

To implement and compare sequential and parallel versions of:

1. Bubble Sort
2. Merge Sort

using OpenMP.

Sequential Bubble Sort Working

Bubble Sort compares adjacent elements and swaps them if they are in wrong order.

Example:

5 3 2 4

Compare 5 and 3 → swap:

3 5 2 4

Compare 5 and 2 → swap:

3 2 5 4

Compare 5 and 4 → swap:

3 2 4 5

Largest element goes to the end after each pass.

Time complexity:

$O(n^2)$

Parallel Bubble Sort Working

Normal Bubble Sort cannot be directly parallelized easily because adjacent swaps depend on each other.

So the program uses **Odd-Even Transposition Sort**, which is a parallel version of Bubble Sort.

It works in phases:

Even Phase

Compares pairs:

(0,1), (2,3), (4,5)

Odd Phase

Compares pairs:

(1,2), (3,4), (5,6)

These pairs are independent, so they can be executed in parallel.

Code:

```
#pragma omp parallel for
```

This allows multiple comparisons at the same time.

Sequential Merge Sort Working

Merge Sort is a divide-and-conquer algorithm.

Steps:

1. Divide array into two halves.
2. Sort left half.
3. Sort right half.
4. Merge both sorted halves.

Example:

[8, 3, 5, 2]

Divide:

[8, 3] and [5, 2]

Further divide:

[8] [3] [5] [2]

Merge sorted:

[3, 8] and [2, 5]

Final merge:

[2, 3, 5, 8]

Time complexity:

$O(n \log n)$

Parallel Merge Sort Working

Merge Sort is suitable for parallelization because left half and right half can be sorted independently.
The program uses:
#pragma omp parallel sections
One section sorts the left half, and another section sorts the right half.
#pragma omp section
parallelMergeSort(arr, left, mid);

#pragma omp section
parallelMergeSort(arr, mid + 1, right);
The code also uses a depth variable.
if (depth <= 3)
This is done to avoid creating too many threads. After a certain depth, sequential merge sort is used.

Main Function Working
The program:

1. Takes number of elements from user.
2. Generates random numbers.
3. Copies same array into 4 arrays:
    - Sequential Bubble Sort
    - Parallel Bubble Sort
    - Sequential Merge Sort
    - Parallel Merge Sort
4. Sorts each array.
5. Checks whether sorted or not using isSorted().
6. Prints execution time.
7. Calculates speedup.

Speedup formula:
Speedup = Sequential Time / Parallel Time

Short Exam Writeup
Parallel Bubble Sort and Merge Sort using OpenMP:
This practical compares sequential and parallel sorting algorithms. Bubble Sort compares adjacent elements and repeatedly swaps them to arrange the array in ascending order. Its parallel version is implemented using Odd-Even Transposition Sort, where even-indexed and odd-indexed pairs are compared in separate phases using OpenMP parallel loops. Merge Sort follows the divide-and-conquer technique by dividing the array into two halves, sorting them recursively, and merging the sorted halves. Parallel Merge Sort uses OpenMP parallel sections to sort left and right halves simultaneously. Execution time of each method is measured using omp_get_wtime(), and speedup is calculated to compare performance.

Viva Points
Q. Why is Bubble Sort slow?
Because it uses nested loops and has $O(n^2)$ time complexity.
Q. What is Odd-Even Transposition Sort?
It is a parallel form of Bubble Sort where independent adjacent pairs are compared in odd and even phases.
Q. Why is Merge Sort better than Bubble Sort?
Merge Sort has $O(n \log n)$ complexity, while Bubble Sort has $O(n^2)$.
Q. Why is Merge Sort suitable for parallelization?
Because left and right halves can be sorted independently.
Q. Why is depth used in parallel merge sort?
To limit thread creation and avoid overhead.
Q. Can parallel sort be slower sometimes?
Yes, for small input sizes, thread overhead may be more than the benefit of parallelism.

Practical 3: Parallel Reduction for Min, Max, Sum, Average

Aim

To perform minimum, maximum, sum, and average operations on an array using sequential and parallel reduction.

Concept of Reduction

Reduction means combining multiple values into a single final result.

Examples:

Sum of array

Minimum element

Maximum element

Average

In parallel programming, multiple threads process different parts of the array. Each thread calculates a partial result, and finally all partial results are combined.

Sequential Working

The sequential part uses one loop:

```
for (int i = 0; i < n; i++) {
   if (arr[i] < seq_min) seq_min = arr[i];
   if (arr[i] > seq_max) seq_max = arr[i];
   seq_sum += arr[i];
}
```

Then average is calculated as:

```
seq_avg = (double)seq_sum / n;
```

Parallel Working

The parallel version uses OpenMP reduction:

```
#pragma omp parallel for reduction(min:par_min) reduction(max:par_max) reduction(+:par_sum)
```

Meaning:

- reduction(min:par_min) finds minimum value.
- reduction(max:par_max) finds maximum value.
- reduction(+:par_sum) calculates sum.

Each thread gets its own private copy of variables. At the end, OpenMP combines them safely.

This avoids race condition.

Why Reduction is Important?

Without reduction, multiple threads may update the same variable at the same time.

Example:

```
sum += arr[i];
```

If many threads update sum, the final answer may become wrong.

Reduction solves this problem by giving each thread a local sum and combining them at the end.

Output Meaning

The program prints:

- First few array elements
- Sequential min, max, sum, average
- Parallel min, max, sum, average
- Sequential time
- Parallel time
- Speedup

If both sequential and parallel results are same, the program is correct.

Short Exam Writeup

Parallel Reduction using OpenMP:

This practical performs min, max, sum, and average operations on a large array using sequential and parallel methods. In the sequential method, a single loop scans the complete array and updates minimum, maximum, and sum values. In the parallel method, OpenMP reduction clauses are used. Each thread calculates local partial results, and OpenMP combines these local values into final results. The clauses reduction(min:par_min), reduction(max:par_max), and reduction(+:par_sum) are used. Average is calculated as sum divided by number of elements. Execution time is measured using omp_get_wtime(), and speedup is calculated.

Viva Points

Q. What is reduction?

Reduction is combining many values into one final result, like sum, min, or max.

Q. Why use reduction in parallel programming?

To avoid race condition while updating shared variables.

Q. What does reduction(+:sum) mean?

Each thread calculates its own sum, and finally all sums are added.

Q. What is speedup?

Speedup shows how much faster the parallel program is compared to sequential program.

Speedup = Sequential Time / Parallel Time

Practical 4: Boston Housing Price Prediction using DNN

Aim

To predict house prices using a **Deep Neural Network** regression model.

Dataset

The program uses the **Boston Housing dataset** from OpenML.

It contains input features related to houses and locations, such as:

- Crime rate
- Number of rooms
- Tax rate
- Pupil-teacher ratio
- Distance from employment centres

The target value is the house price.

X = boston.data

y = boston.target.astype(float)

Here:

- X contains input features.
- y contains output house prices.

Train-Test Split

The dataset is divided into training and testing data.

train_test_split(X, y, test_size=0.2)

This means:

- 80% data is used for training.
- 20% data is used for testing.

Training data is used to teach the model. Testing data is used to check final performance.

Standardization

The features have different ranges. For example, crime rate and tax values may be on different scales.

So, StandardScaler is used:

StandardScaler()

It converts data into a standard scale with mean 0 and standard deviation 1.
This helps neural networks train faster and better.

Model Architecture
The DNN model is:
Dense(64, activation='relu')
Dense(32, activation='relu')
Dense(1)
Explanation:

- First hidden layer has 64 neurons.
- Second hidden layer has 32 neurons.
- Output layer has 1 neuron because we predict one continuous value: house price.

The output layer has no activation because this is a regression problem.

Compilation
model.compile(optimizer='adam', loss='mse', metrics=['mae'])

- adam: optimizer used to update weights.
- mse: Mean Squared Error loss.
- mae: Mean Absolute Error metric.

For regression, MSE and MAE are commonly used.

Training
model.fit(X_train, y_train, validation_split=0.2, epochs=100, batch_size=16)
Meaning:

- Model trains for 100 epochs.
- Batch size is 16.
- 20% of training data is used for validation.

Validation data helps check whether the model is overfitting.

Evaluation Metrics
The program calculates:
MSE
Mean Squared Error. Lower value is better.
RMSE
Root Mean Squared Error. It is easier to understand because it is in the same unit as output.
MAE
Average absolute difference between actual and predicted values.
R^2 Score
Shows how well the model explains variation in target values. Closer to 1 is better.

Graphs
Training vs Validation Loss
If both losses decrease, model is learning well.
If training loss decreases but validation loss increases, model is overfitting.
Actual vs Predicted Scatter Plot
If points are close to diagonal pattern, predictions are good.

Short Exam Writeup
Boston Housing Price Prediction using DNN:
This practical uses a Deep Neural Network to predict house prices from the Boston Housing dataset. The dataset is loaded using fetch_openml, where input features are stored in X and target house prices are stored in y. The data is divided into training and testing sets using train_test_split. Features are standardized using

StandardScaler to bring all values to the same scale. A neural network is built using Keras Sequential model with two hidden Dense layers using ReLU activation and one output neuron for regression. The model is compiled using Adam optimizer, MSE loss, and MAE metric. After training, the model is evaluated using MSE, RMSE, MAE, and R^2 score. Graphs are plotted for training-validation loss and actual versus predicted prices.

Viva Points
Q. Why is this regression?
Because output is a continuous value, house price.
Q. Why no activation in output layer?
Because regression output can be any continuous value.
Q. Why use StandardScaler?
To bring all input features to a similar scale.
Q. Why use MSE?
MSE penalizes large prediction errors.
Q. What is MAE?
It is the average absolute error between actual and predicted values.
Q. What does R^2 score show?
It shows how well the model fits the data.

Practical 5: IMDB Movie Review Classification using DNN
Aim
To classify IMDB movie reviews as **positive** or **negative** using a Deep Neural Network.

Dataset
The IMDB dataset contains movie reviews.
Labels are:
1 = Positive review
0 = Negative review
The dataset is loaded using:
imdb.load_data(num_words=10000)
num_words=10000 means only the top 10,000 most frequent words are used.
The reviews are already converted into numbers. Each number represents a word index.
Example:
[1, 14, 22, 16, 43, ...]
This is not actual text; it is encoded text.

Vectorization
Neural networks cannot directly understand lists of different lengths.
So the program converts each review into a fixed-size vector of length 10,000.
results[i, seq] = 1.0
This is called **multi-hot encoding**.
If a word is present in the review, its position becomes 1. Otherwise, it remains 0.
Example:
Review contains word indexes: [3, 8, 10]

Vector:
0 0 0 1 0 0 0 0 1 0 1 ...

Validation Set
The training data is divided into:

- Actual training data
- Validation data

X_val = X_train[:10000]
partial_X_train = X_train[10000:]
Validation data is used to check model performance during training.

Model Architecture
Dense(64, activation='relu')
Dropout(0.3)
Dense(32, activation='relu')
Dropout(0.3)
Dense(1, activation='sigmoid')
Explanation:

- First hidden layer has 64 neurons.
- Dropout randomly removes 30% neurons during training to reduce overfitting.
- Second hidden layer has 32 neurons.
- Output layer has 1 neuron with sigmoid activation.

Sigmoid gives output between 0 and 1.
Output close to 1 = positive
Output close to 0 = negative

Compilation
model.compile(optimizer='adam', loss='binary_crossentropy', metrics=['accuracy'])

- adam: optimizer
- binary_crossentropy: used for binary classification
- accuracy: measures correct predictions

Prediction
The model gives probability.
Example:
0.87 = positive
0.23 = negative
Usually:
If prediction >= 0.5 → Positive
If prediction < 0.5 → Negative

Graphs
Training vs Validation Accuracy
If both increase, model is learning well.
Training vs Validation Loss
If validation loss increases while training loss decreases, overfitting is happening.

Short Exam Writeup
IMDB Movie Review Classification using DNN:
This practical performs binary classification of IMDB movie reviews into positive and negative classes. The dataset is loaded using Keras IMDB dataset, where reviews are represented as word indexes and labels are 0 or 1. Only the top 10,000 frequent words are considered. Since reviews have different lengths, they are converted into fixed-size 10,000-dimensional multi-hot vectors. A validation set is created from the training data. The DNN model contains Dense layers with ReLU activation and Dropout layers to reduce overfitting. The output layer uses sigmoid activation because the task is binary classification. The model is compiled using Adam optimizer, binary crossentropy loss, and accuracy metric. Finally, test accuracy is calculated and graphs for accuracy and loss are plotted.

Viva Points

Q. Why is this binary classification?
Because there are only two classes: positive and negative.
Q. Why sigmoid in output layer?
Because sigmoid gives probability between 0 and 1.
Q. Why binary crossentropy?
It is suitable for two-class classification problems.
Q. What is Dropout?
Dropout randomly ignores some neurons during training to reduce overfitting.
Q. What is vectorization?
Converting text data into numerical vectors.
Q. What does num_words=10000 mean?
Only the top 10,000 most frequent words are used.

Practical 6: Google Stock Price Prediction using RNN
Aim
To predict Google stock opening price using a Recurrent Neural Network.

Dataset
The program uses Google stock price data.
It loads:

- Training data
- Testing data

train_df = pd.read_csv(train_url)
test_df = pd.read_csv(test_url)
Only the Open price column is used.
training_set = train_df[['Open']].values
real_stock_price = test_df[['Open']].values
The goal is to predict future opening stock price based on previous prices.

Why RNN?
Stock price data is time series data.
In time series, previous values affect future values.
Normal DNN does not remember previous time steps. RNN has memory, so it is suitable for sequence data.

Scaling
The stock prices are scaled between 0 and 1 using MinMaxScaler.
MinMaxScaler(feature_range=(0, 1))
This helps the RNN train better because neural networks perform better with small normalized values.

Creating 60-Day Sequences
The model uses previous 60 days of stock prices to predict the next day price.
Example:
Day 1 to Day 60 → Predict Day 61
Day 2 to Day 61 → Predict Day 62
Day 3 to Day 62 → Predict Day 63
Code:

```
for i in range(60, len(training_set_scaled)):
    X_train.append(training_set_scaled[i-60:i, 0])
    y_train.append(training_set_scaled[i, 0])
```

Input Shape for RNN
RNN expects 3D input:

(samples, timesteps, features)
In this practical:
samples = number of training examples
timesteps = 60 days
features = 1 because only Open price is used
So reshaping is done:
X_train = np.reshape(X_train, (X_train.shape[0], X_train.shape[1], 1))

Model Architecture
SimpleRNN(50, return_sequences=True)
Dropout(0.2)

SimpleRNN(50, return_sequences=True)
Dropout(0.2)

SimpleRNN(50)
Dropout(0.2)

Dense(1)
Explanation:

- Three SimpleRNN layers are used.
- Each RNN layer has 50 units.
- Dropout 0.2 means 20% neurons are ignored during training to reduce overfitting.
- Final Dense layer gives one predicted price.

return_sequences=True is used when another RNN layer comes after the current RNN layer.
The last RNN layer does not need return_sequences=True.

Compilation and Training
model.compile(optimizer='adam', loss='mean_squared_error')
Since this is regression, MSE is used.
Training:
model.fit(X_train, y_train, epochs=25, batch_size=32, validation_split=0.1)

Test Data Preparation
For predicting test data, the program combines train and test stock prices.
Why?
Because to predict the first test day, we need previous 60 days, which are present in training data.
dataset_total = pd.concat((train_df['Open'], test_df['Open']), axis=0)
Then last 60 values before test data are taken and converted into sequences.

Prediction
The model predicts scaled values, so inverse transform is used:
predicted_stock_price = scaler.inverse_transform(predicted_stock_price)
This converts prediction back into original stock price scale.

Evaluation
The program calculates:

- MSE
- RMSE
- MAE

It also plots:

- Actual stock price

- Predicted stock price
- Training loss vs validation loss

If predicted line follows actual line closely, model is performing well.

Short Exam Writeup

Google Stock Price Prediction using RNN:

This practical predicts Google stock opening price using a Recurrent Neural Network. The dataset is loaded using Pandas, and only the Open price column is selected. Since stock data is time-series data, previous values are important for predicting future values. Therefore, an RNN is used. The data is scaled between 0 and 1 using MinMaxScaler. The model uses previous 60 days of stock prices to predict the next day price. The input is reshaped into 3D format as required by RNN: samples, timesteps, and features. The model contains three SimpleRNN layers with Dropout layers to reduce overfitting and one Dense output layer. It is compiled using Adam optimizer and MSE loss. After training, predictions are inverse-transformed to original price scale and evaluated using MSE, RMSE, and MAE.

Viva Points

Q. Why use RNN for stock prediction?

Because stock price is time-series data, and RNN can learn patterns from previous time steps.

Q. Why use 60 timesteps?

The model uses previous 60 days of prices to predict the next day price.

Q. Why MinMaxScaler is used?

To scale stock prices between 0 and 1 for better neural network training.

Q. What is return_sequences=True?

It returns the full sequence output to pass it to the next RNN layer.

Q. Why inverse transform is used?

Because predictions are in scaled form, so inverse transform converts them back to actual stock prices.

Q. Why use MSE loss?

Because stock price prediction is a regression problem.

Common HPC Explanation to Remember

For all HPC programs, compilation is done using:

g++ -fopenmp filename.cpp -o output

-fopenmp enables OpenMP support.

Common OpenMP directives:

#pragma omp parallel

Creates parallel region.

#pragma omp parallel for

Divides loop iterations among multiple threads.

#pragma omp critical

Allows only one thread at a time to execute a section.

#pragma omp section

Used to divide independent code blocks among threads.

reduction(+:sum)

Used to safely combine partial results from multiple threads.

Common Deep Learning Explanation to Remember

All deep learning practicals follow this flow:

Load dataset

Preprocess data

Split or prepare train-test data

Build model

Compile model

Train model
Evaluate model
Plot results
Important terms:
Epoch: One complete pass over the training data.
Batch size: Number of samples processed before updating weights.
Optimizer: Algorithm used to update weights. Example: Adam.
Loss function: Measures error between actual and predicted output.
Activation function: Adds non-linearity to neural network.
Validation data: Data used during training to check model generalization.
Overfitting: When model performs well on training data but poorly on unseen data.

2-Minute Combined Explanation for External
Sir/Madam, I have performed six practicals, three from HPC and three from Deep Learning. In HPC, I used OpenMP with C++ to parallelize graph traversal, sorting, and reduction operations. In the BFS and DFS practical, the graph is represented using an adjacency list. Sequential BFS uses a queue, DFS uses a stack, and the parallel version uses OpenMP directives with critical sections to avoid race conditions. In the sorting practical, I implemented sequential and parallel Bubble Sort and Merge Sort. Parallel Bubble Sort uses odd-even transposition, while Parallel Merge Sort uses parallel sections to sort left and right halves simultaneously. In the reduction practical, I calculated min, max, sum, and average using OpenMP reduction clauses.
In Deep Learning, I used TensorFlow and Keras. The first model predicts Boston house prices using a DNN regression model. The second classifies IMDB reviews as positive or negative using a binary classification DNN with sigmoid output. The third predicts Google stock prices using RNN because stock prices are time-series data. Each model follows the steps of loading data, preprocessing, building the model, compiling, training, evaluating, and plotting results.

Most Important Last-Minute Viva Questions
Q. What is OpenMP?
OpenMP is an API used for shared-memory parallel programming in C/C++ and Fortran.
Q. What is race condition?
Race condition occurs when multiple threads access and update the same variable at the same time, causing incorrect output.
Q. How do you avoid race condition?
Using critical, atomic, locks, or reduction.
Q. What is speedup?
Speedup compares sequential and parallel execution time.
Speedup = Sequential Time / Parallel Time
Q. What is DNN?
A Deep Neural Network is a neural network with multiple hidden layers.
Q. What is RNN?
A Recurrent Neural Network is used for sequence data because it can remember previous time steps.
Q. Difference between regression and classification?
Regression predicts continuous values, while classification predicts categories.
Q. Why sigmoid is used in binary classification?
Because it gives probability between 0 and 1.
Q. Why ReLU is used?
ReLU introduces non-linearity and helps the model learn complex patterns.
Q. Why Dropout is used?
Dropout reduces overfitting by randomly disabling some neurons during training.
Q. Why scaling is needed?
Scaling brings features into similar range and improves training speed and stability.

Practical 1: Parallel BFS and DFS using OpenMP

Algorithm: Parallel BFS

1. Start.
2. Create graph using adjacency list.
3. Initialize visited[] array as false.
4. Insert starting node into frontier.
5. Mark starting node as visited.
6. While frontier is not empty:
    - Create empty next_frontier.
    - Process all nodes of current frontier in parallel.
    - For each neighbour:
        - If not visited, mark it visited inside critical section.
        - Add it to next_frontier.
7. Replace frontier with next_frontier.
8. Print BFS traversal.
9. Stop.

Flowchart: Parallel BFS

Start
↓
Create graph using adjacency list
↓
Initialize visited[] = false
↓
Insert start node into frontier
↓
Mark start node visited
↓
Is frontier empty?
↓ No
Process current frontier nodes in parallel
↓
Check each neighbour
↓
If neighbour not visited
↓
Mark visited using critical section
↓
Add neighbour to next_frontier
↓
frontier = next_frontier
↓
Repeat
↓ Yes
Print BFS traversal
↓
Stop

Algorithm: Parallel DFS

1. Start.
2. Create graph using adjacency list.
3. Initialize visited[] array as false.
4. Start DFS from source node.

5. Check and mark current node as visited inside critical section.
6. Print current node.
7. Explore all unvisited neighbours in parallel.
8. Recursively call DFS for each neighbour.
9. Stop when all reachable nodes are visited.
10. Stop.

Flowchart: Parallel DFS

Start
↓
Create graph using adjacency list
↓
Initialize visited[] = false
↓
Call DFS(start)
↓
Is node already visited?
↓ Yes
Return
↓ No
Mark node visited using critical section
↓
Print node
↓
Process neighbours in parallel
↓
For each unvisited neighbour
↓
Call DFS(neighbour)
↓
All neighbours processed?
↓
Return
↓
Stop

Practical 2: Parallel Bubble Sort and Merge Sort using OpenMP

Algorithm: Parallel Bubble Sort / Odd-Even Sort

1. Start.
2. Input array size and elements.
3. Repeat for n phases:
    - If phase is even:
        - Compare pairs (0,1), (2,3), (4,5) in parallel.
    - If phase is odd:
        - Compare pairs (1,2), (3,4), (5,6) in parallel.
4. If left element is greater than right element, swap them.
5. Continue until all phases complete.
6. Print sorted array.
7. Stop.

Flowchart: Parallel Bubble Sort

Start
↓
Input array

↓
Set phase = 0
↓
phase < n?
↓ No
Print sorted array
↓
Stop
↓ Yes
Is phase even?
↓ Yes
Compare even indexed pairs in parallel
↓
Swap if needed
↓
phase++
↓
Repeat
↓ No
Compare odd indexed pairs in parallel
↓
Swap if needed
↓
phase++
↓
Repeat

Algorithm: Parallel Merge Sort

1. Start.
2. Input array.
3. Divide array into two halves.
4. Sort left half and right half in parallel using OpenMP sections.
5. Continue dividing until single element remains.
6. Merge sorted left and right halves.
7. Repeat until complete array is sorted.
8. Print sorted array.
9. Stop.

Flowchart: Parallel Merge Sort

Start
↓
Input array
↓
Call mergeSort(left, right)
↓
left < right?
↓ No
Return
↓ Yes
Find mid
↓
Sort left half in parallel
↓

Sort right half in parallel
↓
Merge both sorted halves
↓
Return sorted array
↓
Print sorted array
↓
Stop

Practical 3: Parallel Reduction for Min, Max, Sum, Average

Algorithm

1. Start.
2. Input array size.
3. Generate or input array elements.
4. Initialize:
    - min_val
    - max_val
    - sum
5. Use OpenMP parallel for loop with reduction.
6. Each thread calculates local min, max, and sum.
7. Combine local results to get final min, max, and sum.
8. Calculate average:

Average = Sum / Number of elements

9. Display min, max, sum, and average.
10. Stop.

Flowchart

Start
↓
Input array size and elements
↓
Initialize min, max, sum
↓
Apply OpenMP parallel for with reduction
↓
Each thread calculates local min, max, sum
↓
Combine all local results
↓
Calculate average = sum / n
↓
Display min, max, sum, average
↓
Stop

Practical 4: Boston Housing Price Prediction using DNN

Algorithm

1. Start.
2. Import required libraries.
3. Load Boston Housing dataset.
4. Separate input features X and target price y.
5. Split data into training and testing sets.

6. Standardize input features using StandardScaler.
7. Build DNN model:
    - Dense layer with ReLU
    - Dense layer with ReLU
    - Output Dense layer
8. Compile model using Adam optimizer and MSE loss.
9. Train model using training data.
10. Predict house prices on test data.
11. Evaluate using MSE, RMSE, MAE, and R^2 score.
12. Plot loss graph and actual vs predicted graph.
13. Stop.

Flowchart

Start
↓
Import libraries
↓
Load Boston Housing dataset
↓
Separate X and y
↓
Split data into train and test
↓
Standardize features
↓
Build DNN model
↓
Compile model
↓
Train model
↓
Predict on test data
↓
Evaluate model
↓
Plot results
↓
Stop

Practical 5: IMDB Movie Review Classification using DNN

Algorithm

1. Start.
2. Import required libraries.
3. Load IMDB movie review dataset.
4. Select top 10,000 frequent words.
5. Convert reviews into numerical vectors.
6. Split training data into training and validation sets.
7. Build DNN model:
    - Dense layer with ReLU
    - Dropout layer
    - Dense layer with ReLU
    - Dropout layer
    - Output layer with sigmoid

8. Compile model using Adam optimizer and binary crossentropy loss.
9. Train model.
10. Test model on test data.
11. Calculate accuracy.
12. Plot accuracy and loss graphs.
13. Stop.

Flowchart

Start
↓
Import libraries
↓
Load IMDB dataset
↓
Convert reviews into vectors
↓
Create training and validation sets
↓
Build DNN model
↓
Compile model
↓
Train model
↓
Evaluate on test data
↓
Display accuracy
↓
Plot accuracy and loss
↓
Stop

Practical 6: Google Stock Price Prediction using RNN

Algorithm

1. Start.
2. Import required libraries.
3. Load Google stock price training and testing datasets.
4. Select Open price column.
5. Scale data using MinMaxScaler.
6. Create input sequences using previous 60 days.
7. Create target value as next day stock price.
8. Reshape input into 3D format:

samples × timesteps × features

9. Build RNN model:
    - SimpleRNN layer
    - Dropout layer
    - SimpleRNN layer
    - Dropout layer
    - SimpleRNN layer
    - Dropout layer
    - Dense output layer
10. Compile model using Adam optimizer and MSE loss.
11. Train model.

12. Prepare test sequences.
13. Predict stock prices.
14. Apply inverse transform to get original price values.
15. Plot actual vs predicted stock price.
16. Stop.

Flowchart

Start
↓
Import libraries
↓
Load stock price dataset
↓
Select Open price column
↓
Scale data using MinMaxScaler
↓
Create 60-day input sequences
↓
Create target next-day price
↓
Reshape data into 3D format
↓
Build RNN model
↓
Compile model
↓
Train model
↓
Prepare test data
↓
Predict stock prices
↓
Inverse transform predictions
↓
Plot actual vs predicted prices
↓
Stop

Very Short Combined Algorithm Pattern for DL Practicals

You can write this if external asks common flow:

Start
↓
Load dataset
↓
Preprocess dataset
↓
Split into training and testing data
↓
Build neural network model
↓
Compile model
↓

Train model
↓
Evaluate model
↓
Plot output graphs
↓
Stop

Very Short Combined Algorithm Pattern for HPC Practicals

Start
↓
Input data / create graph or array
↓
Run sequential algorithm
↓
Run parallel algorithm using OpenMP
↓
Measure execution time
↓
Compare outputs
↓
Calculate speedup
↓
Display result
↓
Stop